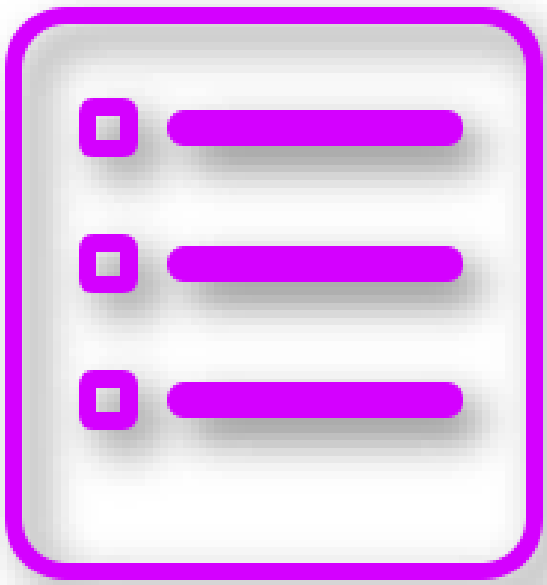


Custom Attributes

Documentation

_____

Creator: Lucas Gomes Cecchini

Pseudonym: AGAMENOM

Overview

This document will help you use **Custom Attributes** for **Unity**.

With it you can customize your **Inspector**.

Just like Unity's **Attributes**, it adds functions that make it easier to use.

Such as easier selection of **Tag** or **Scene name**, **viewing properties**, etc.

Instructions

You can get more information from the Playlist on YouTube:

https://www.youtube.com/playlist?list=PL5hnfx09yM4I_6OdJvShZ0rRtYF9jv6Cd

Script Explanations

Attribute Inspector

What This Script Does

This script is the heart of the package. It replaces Unity's default Inspector for every `MonoBehaviour` and every `ScriptableObject` and draws all the **meta attributes** (ShowIf, ReadOnly, Required, FoldoutGroup, Button, ShowInInspector, DragAndDrop, etc.) in one place. When a component has none of these attributes, the script simply falls back to Unity's default Inspector, so nothing changes for scripts that do not use the package.

The Classes

- **MonoBehaviourInspector** – Custom editor registered for `MonoBehaviour` (and every class that inherits from it). Supports multi-object editing.
- **ScriptableObjectInspector** – The same custom editor registered for `ScriptableObject` assets.
- **AttributeInspector** – The abstract class that contains all the logic. The two classes above are empty and only inherit from it.

Method: OnEnable

Reads and caches the metadata of the inspected type (buttons, Inspector-only members and whether any field has attributes). The reflection work is done only once per type, not on every repaint.

Method: OnInspectorGUI

Decides what is drawn and in which order:

- If the type has no attributes from this package, Unity's default Inspector is drawn.
- Otherwise it draws, in this order: buttons with `Position = Top`, the fields, the `[ShowInInspector]` members and finally the buttons with `Position = Bottom`.
- The serialized object is updated before drawing and the changes are applied at the end.

Fields: DrawFields and DrawGroup

All visible serialized properties are collected in Unity's normal order. Fields without a group are drawn in place. The first field of a `[FoldoutGroup]` decides where the whole group appears, and every field with the same group name is drawn together inside one foldout. A group whose fields are all hidden is not drawn. The open/closed state of each foldout is saved with `SessionState`, so it survives repaints without being saved in the scene or asset.

Method: DrawProperty

Draws one field and applies every attribute that belongs to it, in this order:

- The `m_Script` field is always drawn disabled.
- Conditions (`ShowIf`, `HideIf`, `EnableIf`, `DisableIf`) are evaluated. A hidden field is not drawn. A condition that cannot be evaluated shows a warning box with the reason.
- `[GetComponent]`, `[GetInChildren]` and `[GetInParent]` fill an empty reference automatically.
- `[Required]` shows an error box and tints the field red when the value is empty.
- `[ReadOnly]` disables the field according to its mode (Always, WhenPlaying, WhenEditing).
- `[GUIColor]` tints the field.
- `[GizmoPath]` adds the path toggle button and `[DragAndDrop]` adds the drop area above the list.
- Fields that are `[Serializable]` classes with attributes inside are drawn recursively, so the attributes also work in nested classes and in arrays of nested classes.
- If the value changed, every `[OnValueChanged]` callback is invoked.

Method: TryAutoReference

Fills an empty object reference with the component found on the same object, in the children or in the parents. It runs only in Edit Mode, with a single object selected, and never overwrites a reference that is already assigned.

Method: InvokeOnChanged

Applies the pending changes, finds the object that owns the field (even inside nested classes or arrays) and calls each callback method. This is done for every selected object.

Method: DrawShowInInspector

Draws the fields and properties marked with `[ShowInInspector]` as read-only values, using Unity's native controls for numbers, strings, vectors, colors, enums and objects. Collections are shown as a count. Members with `OnlyInPlayMode` appear only while playing. If a getter throws an exception, the message is shown in an error box instead of breaking the Inspector. While playing, the Inspector repaints constantly so the values stay updated.

Buttons: DrawButtons, DrawButton, Invoke and Run

- A button is enabled according to its `Mode` (Always, PlayModeOnly, EditModeOnly). Methods that return `IEnumerator` are always Play Mode only.
- Methods with parameters get a box with one field for each parameter. Supported types: int, float, double, bool, string, Vector2, Vector3, Color, enums and Unity objects. Other types show a warning and disable the button.
- Parameter values are remembered between repaints. Default values declared in the method are used as initial values.
- `Confirm = true` opens a confirmation dialog first. `Height` changes the height of the button.
- The call is registered in Unity's Undo system. Static methods run once, instance methods run once for each selected object.
- A returned coroutine is started on the `MonoBehaviour`. Exceptions are sent to the Console, and the object is marked as dirty so prefab overrides and scene changes are saved.

Usage

Nothing needs to be configured. Just use the attributes in your scripts:

```
[Required]
public Transform target;

[Button("Reset Health", Confirm = true)]
private void ResetHealth() { }
```

Condition Evaluator

What This Script Does

Evaluates the conditions of `ShowIf`, `HideIf`, `EnableIf` and `DisableIf`. It receives the field being drawn and the attribute, and answers one question: do the conditions pass? It is an internal class used by the Attribute Inspector.

Method: TryEvaluate

Goes through every condition name of the attribute and combines the results with the chosen logic:

- **And** – starts as true and every condition must pass.
- **Or** – starts as false and one passing condition is enough.

It returns false (and an error message) as soon as one condition cannot be evaluated, so the Inspector can show the problem instead of a misleading result.

Method: TryEvaluateOne

- Rejects empty names (`Empty condition name.`).
- A leading `!` negates the condition, for example `"!isReady"`. Spaces around the name and the `!` are ignored.
- First it looks for a **sibling serialized property** with that name (same class, also inside nested classes and arrays). This uses the value shown in the Inspector.
- If no sibling exists, it looks for a **field, property or method** with that name in the object that owns the field. The method must have no parameters and return a value.
- If nothing is found, the error `Condition 'name' was not found` is returned.

Method: TryEvaluateProperty

Defines what "true" means for each serialized type when no comparison value is given:

- Boolean – the value itself.
- Enum, Integer and LayerMask – different from zero.
- Float – different from zero.
- String – not empty.
- Object reference – not null (Unity's null check, so destroyed objects count as null).

When a comparison value is given (for example `[ShowIf(nameof(mode), Mode.Advanced)]`), the property is compared with that value instead. Other property types are not supported and produce an error message.

Methods: EvaluateValue, IsTruthy, CompareEquals and TryToDouble

- **IsTruthy** – truthiness of plain C# values: null is false, bool is itself, empty string is false, collections are true when they have items, enums are true when not zero, numbers are true when not zero.
- **CompareEquals** – compares two values. Null (including destroyed Unity objects) only equals null. Booleans and strings use normal equality. Numbers are compared as doubles with a tolerance, so `5` equals `5f`.
- **TryToDouble** – converts numeric values so different number types can be compared.

Usage Examples

```
[ShowIf(nameof(useExtras))]
[HideIf("!useExtras")]
[ShowIf(nameof(mode), Mode.Advanced)]
[ShowIf(ConditionLogic.Or, nameof(useExtras), "!isSimple")]
[DisableIf("!useExtras")]
```

Meta Info

What This Script Does

Reads the attributes of a class through reflection and keeps the result in a cache. Reading attributes is slow, and the Inspector repaints many times per second, so every result is calculated once and reused. It contains three internal classes.

Class: MetaInfo (field metadata)

Stores everything the package needs to know about **one field**:

- `Conditions` – every ShowIf / HideIf / EnableIf / DisableIf of the field (a field can have several).
- `OnChanged` – every OnValueChanged callback.
- `ReadOnly`, `Required`, `Color`, `AutoRef`, `Group`, `Path` and `DragDrop` – the single attributes of the field.
- `HasPropertyAttribute` – true when the field has a Unity `PropertyAttribute` (such as TagDropdown or MinMaxSlider). These fields keep their own drawer.
- `HasAny` – true when the field has anything that the custom Inspector must process.

For(field) returns the cached metadata, or builds it the first time. Fields without any attribute share one static `Empty` object, so no memory is wasted. If the same single attribute appears twice, the last one is used.

Class: ButtonInfo (button metadata)

Stores everything about **one method** marked with `[Button]`: the method, the attribute, its parameters, the label, whether it is a coroutine and whether all parameters have a supported type. The label comes from the attribute or from the method name, formatted to look like Unity's field names. Generic methods cannot be called and are marked as unsupported.

Class: TypeFeatures (type metadata)

Stores what a whole class offers: its buttons, its `[ShowInInspector]` members and whether any field needs custom drawing.

- **For(type)** – cached lookup per type.

- **Build** – walks the class and its base classes (stopping at Unity's own classes such as `MonoBehaviour`). Buttons of a base class appear before the buttons of a derived class, and buttons of the same class keep the order of the code. Indexers are ignored.
- **FormatName** – turns names such as `resetHealth`, `reset_health` or `XMLParser` into `Reset Health` and `XML Parser`.
- **NestedHasMeta** – checks if a `[Serializable]` class has fields with attributes, up to 3 levels deep. Unity and System types and `UnityEngine.Object` types are ignored. This is what allows attributes to work inside nested classes.

Why It Matters

- Faster Inspector – reflection is done once, not every repaint.
- A single place that decides which attributes exist and how they are grouped.

Property Reflection

What This Script Does

A set of reflection tools that connect Unity's `SerializedProperty` (what the Inspector shows) with the real C# field (what the code uses). It is an internal static class used by the Inspector, the drawers and the gizmo tools. Field lookups are cached.

Methods

- **FindField(type, name)** – finds a field by name, one inheritance level at a time. A field of a derived class wins over a field with the same name in a base class. Private and static fields are included.
- **GetElementType(type)** – returns the item type of an array or `List<T>`, or null.
- **GetFieldInfo(property)** – finds the C# field that a serialized property represents. Array indices such as `Array.data[3]` are removed from the path, so it also works inside arrays and nested classes. Results (even failures) are cached.
- **GetOwner(target, path)** – returns the object that **contains** the field, for example the nested class instance or the array element. Needed to read values and call methods.
- **FindSibling(property, name)** – finds another serialized property next to the current one (same nested class or array element). If none exists there, it searches from the root. Used by conditions and by the gizmos to find linked fields such as the size of a cube.
- **TryGetMember(owner, name, out value)** – reads a field, a property or a method (without parameters) by name. Exceptions are written to the Console and the method returns false.
- **TryRead<T>(owner, name, out result)** – the same, but only succeeds when the value has the expected type.
- **InvokeCallback(owner, field, methodName)** – calls the method used by `[OnValueChanged]`. The method may have no parameters or one parameter; with one parameter, it receives the new value of the field. If it is not found, a warning is shown in the Console.

Constants

- `AllMembers` – the reflection flags (public, private, instance, static) shared by the whole package.

Drag And Drop Area

What This Script Does

Draws the drop area of the `[DragAndDrop]` attribute above an array or `List` of Unity objects and adds the dropped objects to the list. It is an internal static class called by the Attribute Inspector.

How Dropped Objects Are Resolved

Each dragged object is compared with the type of the list items:

- **Direct match** – the object already has the right type.
- **GameObject or Component** – the matching component is taken from it.
- **Sub-assets** – for example, dropping a Texture adds the Sprites inside it.
- **Folders** – every asset of the right type inside the folder is added (when `AcceptFolders` is true).

Methods

- **Draw** – reserves the area, draws it and handles the drag events. The area is disabled while several objects are selected.
- **DrawArea** – draws the box and its text. The text comes from the attribute or is generated from the item type. The box is highlighted while a valid drag is over it.
- **HandleDragEvents** – shows the "copy" cursor for valid objects and accepts the drop.
- **CanAccept and Collect** – decide whether the dragged objects can be used and build the list of objects to add. Duplicates are removed.
- **AddToList** – empty (null) slots are filled first, then the list grows. If `AllowDuplicates` is false, objects already in the list are skipped.

Options of the Attribute

- `Label` – text of the area.
- `Height` – height in pixels (default 60).
- `AllowDuplicates` – allows the same object twice (default false).
- `AcceptFolders` – accepts project folders (default true).

Usage

```
[DragAndDrop("Drop Materials here")]
public List<Material> materials;

[DragAndDrop(Height = 40f, AllowDuplicates = true)]
public List<Sprite> sprites;
```

Decorator Drawers

What This Script Does

Contains the drawers of the decorator attributes. A decorator is drawn **above** a field and does not change the field itself. Each drawer tells Unity how tall it is (`GetHeight`) and how to draw it (`OnGUI`).

InfoBoxDrawer – [InfoBox]

- The height is calculated from the message and the width of the Inspector, so long messages wrap correctly. The minimum height is 32 pixels.
- Draws a Unity help box. `InfoBoxType` selects the icon: Info, Warning, Error or None.

TitleDrawer – [Title]

- Draws the text in bold with a thin gray line under it.

HorizontalLineDrawer – [HorizontalLine]

- Draws a line with the thickness and color given in the attribute. The height of the space is the thickness plus a small margin.

Usage

```
[InfoBox("Configure the character.", InfoBoxType.Warning)]
[Title("Movement")]
[HorizontalLine(2f, "#FF0000FF")]
public float speed;
```

Dropdown Drawers

What This Script Does

Contains every dropdown of the package: tags, scenes, layers and dropdowns whose options come from your own code.

Shared Classes

- **ChoiceSet** – holds the list of options already converted to `GUIContent`, so they are not created again at every repaint. It also keeps a version with an extra first item used as a placeholder.
- **StringChoiceDrawer** – abstract base of the tag and scene drawers. It draws the popup, keeps an invalid saved value visible as a placeholder (index 0), shows a help box below the field when the value is invalid, supports multi-object editing and only accepts string fields. Selecting the placeholder never changes the value.

TagDropdownDrawer – [TagDropdown]

- Lists the tags of the project. The list is refreshed at most once per second.
- Missing tag: the placeholder shows `Tag Missing (value)` and a warning box appears.

SceneTagDropdownDrawer – [SceneTagDropdown]

- Lists the **enabled** scenes of the Build Settings. The value stored is the scene name, not the path.
- The cache is rebuilt automatically when the Build Settings change.
- Three messages: no scene selected (warning), scene disabled in the Build Settings (warning) and scene not found (error).

LayerDropdownDrawer – [LayerDropdown]

- Works with `int` (shows `number: name`) and with `string` (shows the name).
- A layer that does not exist shows `Layer Missing (value)`.

DropdownDrawer – [Dropdown("MemberName")]

- Gets the options from a field, property or method of the same class that returns an `IEnumerable`.
- Works with `string`, `int` and `float` fields.
- If the member is not found, the field stays editable and an error explains the problem. If the current value is not an option, the placeholder `Value Missing` and a warning are shown.

Usage

```
[TagDropdown] public string tagName;
[SceneTagDropdown] public string sceneName;
[LayerDropdown] public int layerIndex;

[Dropdown(nameof(Difficulties))]
public string difficulty = "Normal";
private string[] Difficulties => new[] { "Easy", "Normal", "Hard" };
```

Reference Drawers

What This Script Does

Contains the drawers for object references: missing references, required interfaces and expandable objects.

HighlightEmptyReferenceDrawer – [HighlightEmptyReference]

- When the reference is empty, the field gets a red background and an error box: `Assign an object of type 'X' here!`. For arrays and lists, the type shown is the type of the items.
- If the reference was assigned before but the object was deleted or destroyed, the message changes to explain that the object is missing.
- It does not warn when several objects with different values are selected.

RequireInterfaceDrawer – [RequireInterface(typeof(IMyInterface))]

- Used on a `UnityEngine.Object` field. When you pick a GameObject or a Component, the drawer looks for the component that implements the interface and assigns it. If none is found, a warning is written to the Console and nothing is assigned.
- If an object that does not implement the interface is already assigned, an error box appears.

ExpandableDrawer – [Expandable]

- Adds a foldout to a reference field (for example a `ScriptableObject`). When opened, the fields of the referenced object are drawn inside a box, and they can be edited directly.

- If the reference is empty or the selection has different values, the normal field is drawn.

Usage

```
[HighlightEmptyReference] public Transform player;
[RequireInterface(typeof(IInteractable))] public Object target;
[Expandable] public ScriptableObject config;
```

Value Drawers

What This Script Does

Contains the drawers that change how a **value** is drawn: its label, a suffix, a range slider and a unique ID.

LabelDrawer – [Label("Text")]

- Replaces the displayed name of the field. The tooltip of the original label is kept.

SuffixDrawer – [Suffix("m/s")]

- Draws a small, semi-transparent text at the right side of the field, for example a unit. The text width is measured, so the space adapts to the suffix.
- Not drawn for fields that use several lines (generic properties).

MinMaxSliderDrawer – [MinMaxSlider(min, max)]

- For `Vector2` and `Vector2Int` fields. X is the lower value and Y is the upper value.
- Draws a number field, a slider and a number field. The values are always limited to the range and X never goes above Y.
- `Vector2Int` values are rounded when saved. Other field types show a message.

UniqueIdDrawer – [UniqueId]

- For `string` fields. An empty field receives a new GUID automatically.
- The ID can be read and copied but not typed. The **New** button creates another ID.
- When several objects with different values are selected, no ID is created automatically.

Usage

```
[Label("Movement Speed")] [Suffix("m/s")]
public float speed;

[MinMaxSlider(0f, 100f)]
public Vector2 damageRange = new(10f, 40f);

[UniqueId]
public string saveId;
```

Gizmo Core

What This Script Does

The base of the Gizmo system. It contains the icons, the drawing helpers, the editing session and the preview of gizmos that stay visible.

GizmoIcons

Unity's built-in icons with tooltips: Move, Rotate, Edit (shape) and Path.

GizmoDraw – drawing helpers

- **Axes** – draws the red (X), green (Y) and blue (Z) axes.
- **Cube** – wire cube with softly colored faces, following position, rotation and scale of the object.
- **Sphere** – three wire circles plus a circle facing the Scene camera, with a light fill.
- **Cone** – cap circle, four radial lines, two arcs and a light fill. The angle is the full opening of the cone.

- **Path** – spheres with the point number, lines between points and an arrow in the middle of each segment. The path can be closed.
- **ToWorld / ToLocal** – convert a point between local space and world space.
- **UniformScale and SafeRotation** – use the largest scale axis, and replace an invalid all-zero quaternion with the identity rotation.

GizmoTool, GizmoButton and GizmoSession

- **GizmoTool** – base class of every editing tool. It has an `Id` and the method `OnScene` that draws the handles.
- **GizmoButton** – describes a button of the Inspector: id, icon and a function that creates the tool.
- **GizmoSession** – makes sure that **only one tool is active at a time**. Clicking a button starts the tool, clicking again stops it. The session stops automatically when: the selection changes, Play Mode starts or ends, scripts are recompiled, or the edited field no longer exists.

GizmoPreview – AlwaysShow

Draws the gizmos of attributes that use `AlwaysShow = true`, even when no tool is active. It only draws for the **selected** objects and skips the field that is being edited. Cube, Sphere, Cone, Transform and Path are supported. The list of fields is cached for each type.

Usage

This script is used by the other gizmo scripts. You only need `AlwaysShow = true` in the attribute to use the preview:

```
[GizmoSphere(nameof(radius), AlwaysShow = true)]
public Vector3 center;
public float radius = 1f;
```

Gizmo Drawers

What This Script Does

Contains the property drawers of the gizmo attributes. They draw the row of buttons in the Inspector, check that the linked fields exist, and start the correct tool when a button is clicked.

GizmoDrawerBase

The abstract class that all gizmo drawers share:

- Only works on `Vector3` fields of a `Component`. Other types show the message `Gizmo attributes require a Vector3 field.`
- **Validate** – each drawer implements it. A message means the setup is wrong and an error box is shown instead of the buttons.
- **CreateButtons** – each drawer implements it to say which buttons exist.
- Draws the buttons (20 x 34 pixels) above the field. A pressed button means that the tool is active.

The Drawers

- **GizmoTransformDrawer** – Move button, and a Rotate button when a rotation field is linked. The rotation field must be a Quaternion or a Vector3.
- **GizmoCubeDrawer** – Size and Move buttons. Needs a linked Vector3 for the size.
- **GizmoSphereDrawer** – Size and Move buttons. Needs a linked float for the radius.
- **GizmoConeDrawer** – Size and Move buttons. Needs two linked floats: angle and distance.

The Path attribute has no drawer here. Its toggle button is drawn by the Attribute Inspector because it works with an array.

Usage

```
[GizmoTransform(nameof(rotation))] public Vector3 position;
public Quaternion rotation;

[GizmoCube(nameof(size), "#00FFFF")] public Vector3 center;
public Vector3 size = Vector3.one;
```

```
[GizmoCone(nameof(angle), nameof(distance))] public Vector3 eye;
public float angle = 90f;
public float distance = 5f;
```

Gizmo Tools

What This Script Does

Contains the tools that run in the Scene View. Each tool receives the component, the edited property and the Scene View, draws the handles and writes the new values back with Undo support. Positions are converted between world space and the object's local space.

GizmoToolUtil

- **HandleRotation** – the handles follow the object's rotation when Unity's pivot is set to Local, and the world axes when it is set to Global.
- **MoveCenter** – shows a position handle and saves the new center.

TransformTool

- **Position mode** – position handle. The result is saved in local space.
- **Rotation mode** – rotation handle for the linked rotation field. Works with Quaternion and with Euler angles (Vector3).
- Draws the axes and the name of the field.

CubeTool

- **Size mode** – box handle with faces to resize the cube. Changes both the size and the center.
- **Move mode** – shows the cube and a position handle for the center.

SphereTool

- **Size mode** – sphere handle that changes the radius. The radius never becomes negative and the tool ignores objects with an almost zero scale.
- **Move mode** – position handle for the center.

ConeTool

- **Size mode** – two handles: one at the tip for the **distance** (only the movement along the forward direction counts) and one at the edge for the **angle**.
- **Move mode** – position handle for the origin.

PathTool

- Works on an array of `Vector3`. The points are saved in local space or in world space depending on `WorldSpace`.
- Click a point sphere to select it and move it with the position handle.
- Click the small dot in the middle of a segment to **insert a new point** there.
- A closed path (`Closed = true`) also connects the last point to the first one.

How It Works Internally (Summary)

- 1. You click a button in the Inspector.
- 2. GizmoSession stores the object and the property path, and starts the tool.
- 3. At each Scene View repaint, the tool draws its handles.
- 4. When a handle moves, the value is converted to local space and saved.
- 5. Unity's Undo and serialization record the change.

Button Attribute

What This Script Does

Defines `[Button]`, which turns a method into a clickable button in the Inspector. The attribute only stores the settings. The Attribute Inspector draws the button and calls the method.

Enums

- **ButtonMode** – `Always` (Edit Mode and Play Mode), `PlayModeOnly` or `EditModeOnly`.
- **ButtonPosition** – `Top` or `Bottom`, relative to the fields.

Properties of ButtonAttribute

- `Label` – text of the button. When omitted, the method name is used, formatted as `My Method`.
- `Mode` – when the button is available (default Always).
- `Position` – above (Top, default) or below (Bottom) the fields.
- `Confirm` – asks for confirmation before running the method.
- `Height` – height of the button. Zero uses Unity's default height.

Supported Methods

- Private, public and static methods.
- Methods with parameters of type int, float, double, bool, string, Vector2, Vector3, Color, enums and Unity objects. Fields to fill them appear above the button.
- Coroutines (methods that return `IEnumerator`). They only run in Play Mode.

Usage

```
[Button]
private void SimpleButton() { }

[Button("Reset Health", Confirm = true)]
private void ResetHealth() { }

[Button(Position = ButtonPosition.Bottom, Height = 28f)]
private void WithParameters(string message = "Hello", int times = 2) { }

[Button(Mode = ButtonMode.PlayModeOnly)]
private IEnumerator Countdown() { yield return null; }
```

Decorator Attributes

What This Script Does

Defines the attributes that change the **appearance** of the Inspector. They do not change values. The drawing is done by the Decorator Drawers and the Value Drawers.

The Attributes

- **[InfoBox(message, type)]** – message box. `InfoBoxType`: None, Info (default), Warning, Error.
- **[Title(text)]** – bold title with a line under it.
- **[HorizontalLine(thickness, hexColor)]** – separator. Thickness has a minimum of 1 and the default color is semi-transparent gray (`#80808080`). An invalid color becomes gray.
- **[Label(text)]** – replaces the name of the field.
- **[Suffix(text)]** – text at the end of the field, such as a unit.

Usage

```
[InfoBox("Configure the character settings.")]
[Title("Movement")]
[HorizontalLine(2f, "#808080FF")]
[Label("Movement Speed")]
[Suffix("m/s")]
public float movementSpeed = 5f;
```

Field Attributes

What This Script Does

Defines the attributes that change **how a field is selected, validated or edited**. They are markers (or hold small settings) and the work is done by the Dropdown, Reference and Value Drawers.

Dropdown Attributes

- **[TagDropdown]** – tag selection for a `string`.
- **[SceneTagDropdown]** – scene selection (Build Settings) for a `string`.
- **[LayerDropdown]** – layer selection for an `int` or a `string`.
- **[Dropdown(memberName)]** – options from a member of your class (`string`, `int` or `float`).

Reference Attributes

- **[HighlightEmptyReference]** – highlights an empty or missing reference.
- **[RequireInterface(typeof(IFoo))]** – the reference must implement the interface.
- **[Expandable]** – shows the content of the referenced object inline.

Value Attributes

- **[MinMaxSlider(min, max)]** – range slider for `Vector2` or `Vector2Int`.
- **[UniqueId]** – automatic unique ID for a `string`.

Usage

```
[TagDropdown] public string targetTag;
[LayerDropdown] public int targetLayer;
[Dropdown(nameof(Options))] public string choice;
[RequireInterface(typeof(IInteractable))] public Object target;
[MinMaxSlider(0f, 100f)] public Vector2 range;
```

Gizmo Attributes

What This Script Does

Defines the attributes that configure the Scene View gizmos. All of them are used on a `Vector3` field (the center or position) and receive the **names** of other fields that complete the shape.

GizmoAttributeBase

- `Color` – color of the gizmo.
- `AlwaysShow` – keeps the gizmo visible while the object is selected, even if no tool is active.
- Colors can be given as RGB values (0 to 1) or as a hexadecimal text such as `"#FF0000FF"`. An invalid hexadecimal text uses the default color of the attribute.

The Attributes

- **[GizmoTransform]** / **[GizmoTransform(rotationName)]** – position gizmo, with optional rotation. Default color: white.
- **[GizmoCube(sizeName)]** – cube. The size is another Vector3. Default color: cyan.
- **[GizmoSphere(radiusName)]** – sphere. The radius is a float. Default color: green.
- **[GizmoCone(angleName, distanceName)]** – cone, for example a field of view. Default color: yellow.
- **[GizmoPath]** – path for an array of `Vector3`. `Closed` connects the last point to the first and `WorldSpace` saves the points in world coordinates. Default color: yellow.

Usage

```
[GizmoCube(nameof(size), "#00FFFF", AlwaysShow = true)]
public Vector3 center;
public Vector3 size = Vector3.one;

[GizmoSphere(nameof(radius), 1f, 0f, 0f)]
public Vector3 sphereCenter;
public float radius = 2f;

[GizmoPath(Closed = true)]
public Vector3[] patrolPoints;
```

Meta Attributes

What This Script Does

Defines the attributes that change the **behavior** of a field in the Inspector: visibility, editing, validation, groups, callbacks and automatic references. They only store information. The Attribute Inspector reads them and applies the behavior.

All of them inherit from `MetaAttribute`, can be applied to fields and properties and, in most cases, several can be combined on the same field.

Conditions

- **[ShowIf]** – shows the field when the condition is true.
- **[HideIf]** – hides the field when the condition is true.
- **[EnableIf]** – enables editing when the condition is true.
- **[DisableIf]** – disables editing when the condition is true.

Three ways to write them:

```
[ShowIf(nameof(isEnabled))]
[ShowIf(nameof(mode), Mode.Advanced)]
[ShowIf(ConditionLogic.Or, nameof(a), "!b")]
```

The name can be a field, a property or a method without parameters. A `!` in front negates the condition. `ConditionLogic.And` needs all conditions to be true, `ConditionLogic.Or` needs only one.

Other Attributes

- **[ReadOnly(mode)]** – `Always` (default), `WhenPlaying` or `WhenEditing`.
- **[Required(message)]** – error box and red tint when the field is empty. The message is optional.
- **[OnValueChanged(methodName)]** – calls the method when the value changes. The method has no parameters or one parameter that receives the new value.
- **[GUIColor(r, g, b, a)]** or **[GUIColor("#hex")]** – colors the field.
- **[FoldoutGroup(name)]** – puts fields in the same foldout. `DefaultExpanded` (default true) sets the initial state.
- **[ShowInInspector(label)]** – shows a property or private field as read-only. `OnlyInPlayMode` shows it only while playing.
- **[GetComponent]**, **[GetInChildren]**, **[GetInParent]** – fill an empty reference automatically. `IncludeInactive` (default true) includes disabled objects.
- **[DragAndDrop(label)]** – drop area for arrays and lists of Unity objects. Options: `Height`, `AllowDuplicates`, `AcceptFolders`.

Enums

- `ConditionLogic` (And, Or), `ConditionEffect` (Show, Hide, Enable, Disable), `ReadOnlyMode` (Always, WhenPlaying, WhenEditing) and `AutoReferenceSource` (Self, Children, Parent).

Usage

```
[FoldoutGroup("Movement")]
[Required("A target is required.")]
public Transform target;

[ReadOnly(ReadOnlyMode.WhenPlaying)]
public float currentSpeed;

[OnValueChanged(nameof(UpdateMovement))]
public float movementSpeed;

[GetComponent]
public Rigidbody body;
```

Test Attributes

What This Script Does

A demonstration component that shows **every attribute of the package** working. Add `TestAttributes` to a GameObject and explore the Inspector. It is also a good reference to copy examples from. The script is in the `CustomAttributes.Samples` namespace.

Sections of the Component

- **Conditions** – ShowIf with enums and booleans, HideIf, an OR condition with a negated property and DisableIf.
- **Validation** – ReadOnly (also WhenPlaying), Required, GUIColor, OnValueChanged and Suffix.
- **Dropdowns & references** – TagDropdown, SceneTagDropdown, LayerDropdown (int and string), Dropdown (from a property and from a method), RequireInterface with the `IInteractable` interface, Expandable, GetComponent / GetInChildren / GetInParent, MinMaxSlider (Vector2 and Vector2Int) and UniqueId.
- **Drag and drop lists** – lists of Materials, Colliders, Sprites and AudioClips with different options.
- **Groups / nested data** – FoldoutGroup "Stats", and the `SpawnPoint` class, which shows that attributes (ShowIf, GizmoSphere, ReadOnly, UniqueId) also work inside nested classes and arrays.
- **Gizmos** – Transform, Cube, Sphere, Cone and Path, all with `AlwaysShow = true`.
- **Buttons / debug members** – ShowInInspector (normal and Play Mode only), a simple button, a button with confirmation, a button with parameters, a coroutine button (Play Mode only) and a static button (Edit Mode only).

Supporting Types

- **IInteractable** – example interface used by `[RequireInterface]`.
- **Mode** – example enum (Simple, Advanced) used by the conditions.
- **SpawnPoint** – example `[Serializable]` class used for the nested data.

How to Use

- 1. Create an empty GameObject and add the `TestAttributes` component.
- 2. Change `mode` and `useExtras` to see fields appear, disappear and become disabled.
- 3. Press the gizmo buttons below the fields and edit the shapes in the Scene View.
- 4. Press the buttons at the end of the Inspector to test the Button attribute.

This component can be removed from a project without affecting the rest of the package.